

Introduction to NLP & Text Analytics

Text Pre-processing · Feature Extraction · HMM · Embeddings

Rahul Basu

Department of Computer Science & Engineering

Session 1

Rahul Basu

What is NLP?

Text
Analytics
Framework

NLP Libraries

Text Pre-
processing

Feature
Extraction

N-grams &
Language
Models

POS Tagging

Named Entity
Recognition

Word
Embeddings
& Word2Vec

Text
Classification

Web Scraping

Summary

Numerical
Exercises

Further

- 1 What is NLP?
- 2 Text Analytics Framework
- 3 NLP Libraries
- 4 Text Pre-processing
- 5 Feature Extraction
- 6 N-grams & Language Models

- 7 POS Tagging
- 8 Named Entity Recognition
- 9 Word Embeddings & Word2Vec
- 10 Text Classification
- 11 Web Scraping
- 12 Summary
- 13 Numerical Exercises

Natural Language Processing

Definition

NLP is a sub-field of Artificial Intelligence that enables computers to **understand, interpret, and generate** human language in a useful way. [8]

Why is NLP hard?

- **Ambiguity** — “I saw the man with a telescope”
- **Context-dependence** — same word, different meaning
- **Coreference** — “She said *she* was tired”
- **Figurative language** — irony, metaphor
- **World knowledge** — implicit

Difficulty scale (1–10)

Task	Rule	ML
Tokenization	2	1
POS Tagging	4	2
NER	5	3
Sentiment	6	4
Coreference	9	7
Reasoning	10	9

Understanding

- Sentiment Analysis
- Named Entity Recognition
- Syntactic Parsing
- Coreference Resolution
- Information Extraction

Generation

- Machine Translation
- Summarisation
- Dialogue / Chatbots
- Text-to-SQL
- Story generation

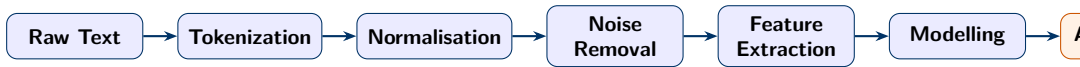
Hybrid

- Question Answering
- Code Generation
- Document Q&A (RAG)
- Caption generation
- Instruction following

Industry Uptake

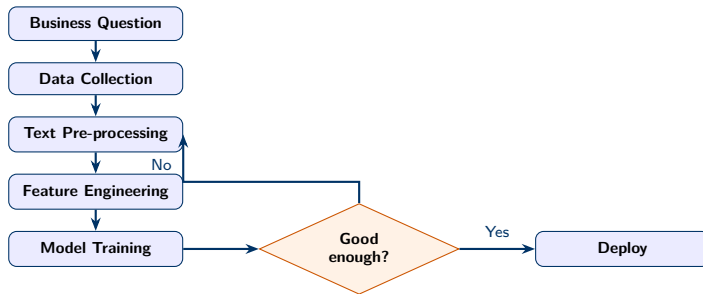
LLM-based NLP products (ChatGPT, Copilot, Gemini) reached >100 M users within months of release — the fastest technology adoption on record. [13]

The Standard NLP Pipeline



Stage	Purpose	Example tools
Tokenization	Split text into atomic units	NLTK, spaCy, HuggingFace
Normalisation	Lowercase, expand contractions	regex, unicodedata
Noise Removal	Strip HTML/URLs/punctuation	BeautifulSoup, regex
Feature Extraction	Convert text to numeric vectors	TF-IDF, Word2Vec, BERT
Modelling	Train statistical/neural model	scikit-learn, PyTorch
Application	Deploy to downstream product	Flask, FastAPI

Text Analytics Lifecycle



Key Insight

The pipeline is **iterative**, not linear. The “good enough?” gate drives repeated cycles of data collection, pre-processing refinement, and model tuning. Most production NLP projects spend 60–80% of effort on data quality.

NLTK [1]

Created 2001 at U. Penn by Bird & Loper.

- 80+ bundled corpora (WordNet, Reuters, Brown)
- Rich stemmer suite (Porter, Lancaster, Snowball)
- Ideal for **teaching** and linguistic research
- Explicit, step-by-step API

spaCy [6]

Created 2015 by Honnibal & Montani (Explosion AI).

- Industrial-strength neural pipeline
- Pre-trained models for 60+ languages
- Dependency parsing, NER, coref out-of-the-box
- **Production** oriented; 10–50× faster

Feature	NLTK	spaCy
POS Tagging	Rule + HMM	Neural (CNN)
Stemming	Porter/Lancaster/Snowball	—
Lemmatization	WordNet lookup	Lookup + rules
NER	MaxEnt/chunker	Neural (transition)
Word Vectors	via Gensim	Built-in (GloVe)
Speed	Moderate	Very fast

Pre-processing Pipeline — Step by Step

Step	What it does	Caution
Lowercasing	Unify “Apple” and “apple”	Skip for NER (case = signal)
Spell correction	Fix “recieve” → “receive”	Slow; may corrupt jargon
URL removal	Strip <code>https://...</code>	Keep for URL classification
HTML stripping	Remove <code><p></code> , <code><div></code>	Use BeautifulSoup
Unicode normalise	é→e, "→" → ASCII	Needed for non-English
Punctuation strip	Remove <code>!.,;?</code>	Keep for sentiment (!)
Tokenize	Split into word units	Choose tokenizer carefully
Stopword removal	Drop “the”, “is”, “at”	Domain stopwords differ
Stem/Lemmatize	“running” → “run”	Over-stemming merges meaning

Pitfall

Removing stopwords before **sentiment analysis** degrades accuracy — words like “not”, “never”, “hardly” are grammatically function words but are semantically critical.


```
from nltk.tokenize import  
    (word_tokenize, TweetTokenizer)  
  
tweet = "OMG! Can't believe #NLP is  
        SO cool"  
  
# Whitespace split  
ws = tweet.split()  
    # 8  
  
# NLTK word_tokenize  
nltk_t = word_tokenize(tweet)  
    # 11  
  
# TweetTokenizer  
tt =  
    TweetTokenizer().tokenize(tweet)  
    # 8
```

Why token counts differ:

- **Whitespace:** splits on spaces only; "Can't" stays whole; no URL handling
- **word_tokenize:** Penn TB rules; splits contractions ("Can't" → ["Ca", "n't"]); '!' separate
- **TweetTokenizer:** preserves #tags, @handles, emoji, URLs as single tokens

Rule of thumb: match tokenizer to domain — TweetTokenizer for social media, word_tokenize for prose.

Stemming

Applies suffix-stripping **rules** mechanically.

- **Porter** (1980) — 5-step algorithm, most common [15]
- **Lancaster** — more aggressive; faster convergence
- **Snowball** — multi-language, improved Porter
- Output may not be a real word: `studies` → `studi`

Lemmatization

Uses POS tag + **dictionary** lookup.

- Requires POS tagging as prerequisite
- Always returns a real dictionary form
- Handles irregular forms: `better` → `good`
- WordNet [12] in NLTK; lookup tables in spaCy

Word	Porter	Lancaster	WordNet Lemma
studies	studi	study	study
running	run	run	run
better	better	bet	good
geese	gees	gees	goose

Lemmatization in Code — WordNet POS Mapping

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
```

```
lemmatizer = WordNetLemmatizer()
```

```
def get_wn_pos(tag):
    if tag.startswith('J'): return
        wordnet.ADJ
    elif tag.startswith('V'): return
        wordnet.VERB
    elif tag.startswith('N'): return
        wordnet.NOUN
    elif tag.startswith('R'): return
        wordnet.ADV
    return wordnet.NOUN # default
```

```
tokens = nltk.word_tokenize("He was
    running quickly")
tagged = nltk.pos_tag(tokens)
# [('He', 'PRP'), ('was', 'VBD'),
#  ('running', 'VBG'), ('quickly', 'RB')]
lemmas = [lemmatizer.lemmatize(w,
```

Without POS (default = NOUN):

Input	Lemma
running	running
was	wa

With POS context:

Input	Lemma
running	run
was	be
quickly	quickly

Always POS-tag first. The default NOUN assumption silently corrupts verbs and adverbs.

Spelling Correction & Emoji Processing

pyspellchecker

```
from spellchecker import SpellChecker
spell = SpellChecker()
text = "I recieve yuor mesage
        today"
misspelled =
    spell.unknown(text.split())
corrected = {w: spell.correction(w)
              for w in misspelled}
# {'recieve': 'receive',
#  'yuor' : 'your',
#  'mesage' : 'message'}
```

Caution: slow on large corpora; corrupts domain jargon ("LSTM"→"last"). Skip for technical or code-mixed text.

emoji — demojize & strip

```
import emoji, re
# raw: "Best movie!
#       [heart][laugh] #NLP"
text = emoji.demojize(text)
# "Best movie! :red_heart:
#   :rolling_on_floor_laughing:
#   #NLP"
text = re.sub(r':[a-z_]+:', '',
              text)
# "Best movie! #NLP"
```

Emoji carry strong sentiment signal. **Demojize first**, then decide: preserve for sentiment analysis, or strip for topic models.

NLTK Corpora — Reuters & Movie Reviews

Reuters Corpus (21,578 news articles)

```
from nltk.corpus import reuters
import pandas as pd

fileids = reuters.fileids()    #
    10,788 docs
text0 = reuters.raw(fileids[0])

# Build a DataFrame for ML
df = pd.DataFrame({
    'text': [reuters.raw(f) for f
              in fileids],
    'cats': [reuters.categories(f)
              for f in fileids]})

# 90 financial categories:
# 'crude', 'earn', 'gold', 'trade'
...
```

Industry standard for topic classification benchmarks.

Movie Reviews Corpus (2,000 reviews)

```
from nltk.corpus import
    movie_reviews as mr

# 1,000 pos + 1,000 neg reviews
reviews, labels = [], []
for cat in mr.categories():
    # ['neg', 'pos']
    for fid in mr.fileids(cat):
        reviews.append(mr.raw(fid))
        labels.append(cat)

from sklearn.model_selection
    import \
        train_test_split
X_tr, X_te, y_tr, y_te =
    train_test_split(
        reviews, labels, test_size=0.2,
        random_state=42)
```

Benchmark for **binary sentiment classification**.

Bag-of-Words & TF-IDF

Bag-of-Words (BoW) [18]

- 1 Build vocabulary V (all unique tokens)
- 2 Represent each document as a count vector of length $|V|$
- 3 Ignores word order; very sparse

Problem: “the”, “is” get large counts but carry no meaning.

TF-IDF [7, 17]

$$\text{TF-IDF}(t, d) = \underbrace{\frac{f_{t,d}}{\sum_k f_{k,d}}}_{\text{TF}} \times \log \underbrace{\frac{N+1}{n_t+1} + 1}_{\text{IDF}}$$

- $f_{t,d}$ — raw frequency of t in d
- N — total documents; n_t — docs containing t
- High IDF $\Rightarrow$ rare, discriminating term
- sklearn uses `sublinear_tf`: $1 + \log f_{t,d}$ to compress outliers

Mini corpus (4 docs, N=4):

- D1: “the cat sat on the mat” D2: “the dog sat on the log”
- D3: “the cat is a lazy animal” D4: “the dog is a brown animal”

Term	n_t (df)	IDF = $\log(5/n_t) + 1$	TF in D1	TF-IDF (D1)
the	4	$\log(1.25) + 1 = 1.22$	$2/6=0.33$	0.41
cat	2	$\log(2.5) + 1 = 1.92$	$1/6=0.17$	0.32
sat	2	$\log(2.5) + 1 = 1.92$	$1/6=0.17$	0.32
mat	1	$\log(5) + 1 = 2.61$	$1/6=0.17$	0.44
dog	2	$\log(2.5) + 1 = 1.92$	0	0

Key Insight

“**mat**” gets the highest TF-IDF in D1 — it appears only there. “**the**” has high TF but near-zero IDF (appears in all docs), so it contributes little discrimination. TF-IDF automatically acts as a soft stopwords filter.

```
from sklearn.feature_extraction.text \
    import CountVectorizer

# Default on movie reviews: 42,251 tokens
# min_df prunes hapax legomena
cv = CountVectorizer(
    stop_words='english',
    min_df=10)          # keep if in
                        # >=10 docs
X = cv.fit_transform(reviews)
print(X.shape)          # (1600, 6740)
print(type(X))
# <class 'scipy.sparse._csr.csr_matrix'>

vocab = cv.get_feature_names_out()
print(vocab[:4])
# ['able' 'absolutely' 'action'
  'actually']
```

min_df	Vocab size
1 (default)	42,251
2	17,880
10	6,740
50	1,843

Vocabulary Pruning

Hapax legomena (once-only words) add noise, not signal. `min_df` silently discards them. Rule of thumb: 2–5 for short texts, 10–50 for long documents.


```
import pandas as pd, numpy as np

# Sparsity: ~0.3% entries are non-zero
sparsity = X.nnz /
    (X.shape[0]*X.shape[1])
print(f"Sparsity: {sparsity:.4f}")    #
    0.0031

# Dense slice for inspection only
df_dtm = pd.DataFrame(
    X[:5].toarray(),
    columns=cv.get_feature_names_out())
print(df_dtm[['action', 'film', 'plot']])
#      action  film  plot
# 0          0     3     1
# 1          1     7     0
# 2          0     4     2
```

Memory comparison (1600 docs × 6740 terms):

Format	Size
Dense float64	≈ 86 MB
CSR sparse	≈ 0.3 MB

sklearn always returns a sparse matrix. Call `.toarray()` only on small slices for visualisation — never on the full corpus matrix.

Common freq. pairs (Reuters):
 “the” (488,142), “” empty (69,224)
 ⇒ always strip empty strings after tokenisation.

```
from sklearn.feature_extraction.text \
    import TfidfVectorizer

tfidf = TfidfVectorizer(
    min_df=5,
    sublinear_tf=True,      # replace tf
                           # with 1+log(tf)
    ngram_range=(1,2))     # unigrams +
                           # bigrams
X_tfidf = tfidf.fit_transform(reviews)
print(X_tfidf.shape)      # (1600, 14621)

# Top-scoring terms for one document
scores = X_tfidf[0].toarray()[0]
top_idx = scores.argsort()[::-1][:5]
vocab = tfidf.get_feature_names_out()
print([vocab[i] for i in top_idx])
# ['film', 'great film', 'story',
#  'highly', 'recommend']
```

Parameter	Effect
sublinear_tf	tf $\rightarrow$ $1 + \log tf$; tames outlier counts
min_df	Prunes rare terms
ngram_range	Adds bigrams; catches "not good"
max_features	Hard vocab cap

sublinear_tf=True almost always helps on long docs — a word appearing 100× is not 100× more informative than one appearing once.

N-grams [19]

Definition

An **n-gram** is a contiguous sequence of n tokens. Captures short-range phrase context invisible to unigrams.

Bigram Language Model:

Examples from “the quick brown fox”:

- Unigram: [the, quick, brown, fox]
- Bigram: [the quick, quick brown, brown fox]
- Trigram: [the quick brown, quick brown fox]

$$P(w_n | w_{n-1}) = \frac{C(w_{n-1}, w_n)}{C(w_{n-1})}$$

Laplace smoothing prevents zero probabilities:

$$P_L(w_n | w_{n-1}) = \frac{C(w_{n-1}, w_n) + 1}{C(w_{n-1}) + |V|}$$

Practical Note

```
from sklearn.feature_extraction.text
    import \
        CountVectorizer

# Bigrams only
bi = CountVectorizer(min_df=10,
                    ngram_range=(2,2))
bi.fit_transform(reviews)
print(bi.get_feature_names_out()[:4])
# ['action comedy', 'action film',
#  'action movie', 'action scenes']
# Trigrams only
tri = CountVectorizer(min_df=10,
                    ngram_range=(3,3))
tri.fit_transform(reviews)
print(tri.get_feature_names_out()[:3])
# ['based true story', 'best film year',
#  'highly recommend film']
# Mixed 1-3 grams
mix = CountVectorizer(ngram_range=(1,3))
```

Why n-grams matter:

- **Bigrams** capture phrases:
“not good”, “highly recommend”,
“action film”
- **Trigrams** capture idioms:
“best film year”, “waste of time”
- Unigrams alone miss negation:
“not” + “good” are treated
independently
⇒ `ngram_range=(1,2)` is the standard
baseline

Adding bigrams roughly **doubles vocab size** —
watch RAM and fit time on large corpora.

Part-of-Speech Tagging

Assigns grammatical labels (Noun, Verb, Adjective ...) to each token.

Penn Treebank Tags (common subset):

Tag	Meaning	Example	Tag	Meaning	Example
NN	Noun, singular	dog	VB	Verb, base	run
NNS	Noun, plural	dogs	VBZ	Verb, 3sg	runs
NNP	Proper noun	London	VBD	Verb, past	ran
JJ	Adjective	quick	IN	Preposition	over
RB	Adverb	quickly	DT	Determiner	the

Why POS matters

Required by lemmatization (“studies” as noun → “study”; “studies” as verb → “study”), syntactic parsing, NER, and information extraction pipelines.

POS Tagging in Code — NLTK & spaCy

NLTK (nltk.pos_tag)

```
import nltk
text = "The cat sits on the mat"
tokens = nltk.word_tokenize(text)
tagged = nltk.pos_tag(tokens)
# [('The', 'DT'), ('cat', 'NN'),
#  ('sits', 'VBZ'), ('on', 'IN'),
#  ('the', 'DT'), ('mat', 'NN')]

# Look up any tag
nltk.help.upenn_tagset('VBZ')
# VBZ: verb, present tense, 3rd
#       person sg
# sits, says, goes, ...
```

spaCy (neural pipeline)

```
import spacy
nlp =
    spacy.load("en_core_web_sm")
doc = nlp("The cat sits on the
    mat")
for token in doc:
    print(token.text, token.pos_,
          token.tag_, token.dep_)

# The    DET    DT    det
# cat    NOUN   NN    nsubj
# sits   VERB   VBZ   ROOT
# on      ADP    IN    prep
# the     DET    DT    det
# mat     NOUN   NN    pobj
```

When to use which

NLTK: quick prototyping and teaching; clean step-by-step control. **spaCy**: production use; a single `nlp(text)` call returns POS tags, dependencies, and NER in one neural pass — 10× faster than NLTK pipelines.

POS Tagging — Three Generations of Models

1. Rule-based (1980s)

- Hand-written patterns
- “word ends in -ing after aux. → VBG”
- Fast, transparent
- Brittle on novel text

2. HMM (1990s) [16]

- Emission: $P(\text{word}|\text{tag})$
- Transition: $P(\text{tag}_i|\text{tag}_{i-1})$
- **Markov assumption:** tag depends only on previous tag
- Viterbi decoding: $O(nT^2)$
- NLTK averaged-perceptron HMM

3. CRF / Neural (2000s+) [9]

- CRF: discriminative, $P(\text{tags}|\text{words})$
- Arbitrary overlapping features
- BiLSTM-CRF: reads left→right and right→left
- BERT fine-tuned: >97% on WSJ [4]

Viterbi Algorithm (HMM)

Dynamic programming finds the **globally best** tag sequence in $O(n \cdot T^2)$ time, where n = sentence length and T = number of tags. Greedy left-to-right decoding is faster but suboptimal.

Markov Property

$$P(q_t | q_1, \dots, q_{t-1}) = P(q_t | q_{t-1})$$

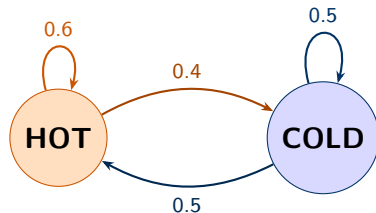
The next state depends **only on the current state**. This collapses an exponential history into one value.

Two-state weather model:

- $a_{HH} = 0.6$: HOT day likely followed by HOT
- $a_{HC} = 0.4$: HOT day sometimes flips to COLD
- $a_{CH} = 0.5$: COLD day equally likely to flip HOT
- $a_{CC} = 0.5$: or stay COLD

Example sequence probability:

$$P(H,C,H) = \pi_H \cdot a_{HC} \cdot a_{CH} = 0.8 \cdot 0.4 \cdot 0.5 = 0.16$$



Rows of the transition matrix A each **sum to 1**:
 $a_{HH} + a_{HC} = 1$, $a_{CH} + a_{CC} = 1$.

Hidden Markov Model — Formal Definition

Five components [16]

$Q = \{q_1, \dots, q_N\}$	hidden states
$A = [a_{ij}]$	$N \times N$ transition matrix
$B = \{b_j(o)\}$	emission distributions
$\pi = [\pi_i]$	initial state distribution
$O = o_1, \dots, o_T$	observation sequence

Two key assumptions:

- 1 **Markov:** $P(q_t \mid q_{t-1}, \dots) = P(q_t \mid q_{t-1})$
- 2 **Output independence:**
 $P(o_t \mid q_t, o_{1:t-1}) = P(o_t \mid q_t)$

Ice cream observable example:

- A **friend in another city** reports how many ice creams they eat each day: 1, 2, or 3.
- The **hidden cause** is the local weather: HOT or COLD.
- We **observe** the sequence of ice cream counts but **never** see the weather directly.

Why “hidden”?

In POS tagging: **words are observed**, **tags are hidden**. The HMM models how a latent tag sequence generates the visible words.

Hidden states: HOT (H), COLD (C) **Observations:** 1, 2, 3 ice creams **Initial:** $\pi_H = 0.8$, $\pi_C = 0.2$

Transition matrix A

	H	C
H	0.6	0.4
C	0.5	0.5

Emission matrix B

	1	2	3
H	0.2	0.4	0.4
C	0.5	0.4	0.1

Joint probability formula

$$P(O, Q) = \pi_{q_1} \cdot b_{q_1}(o_1) \prod_{t=2}^T a_{q_{t-1}q_t} \cdot b_{q_t}(o_t)$$

Reading B

$b_H(3) = 0.4$: given HOT weather, 40% chance of eating 3 ice creams.

$b_C(3) = 0.1$: given COLD weather, only 10% chance of eating 3.

Three Fundamental HMM Problems

Problem 1 — Likelihood [16]

Given model $\lambda = (A, B, \pi)$ and observation sequence O , compute $P(O | \lambda)$.

Application: Which of several HMM models best explains a given sequence? (ASR acoustic scoring)

Problem 2 — Decoding

Given λ and O , find the **best** hidden state sequence $Q^* = \arg \max_Q P(Q | O, \lambda)$.

Application: POS tagging, speech recognition, gene finding. **Solved by the Viterbi algorithm.**

Problem 3 — Learning

Given O (and possibly structure of λ), find parameters $\lambda^* = \arg \max_{\lambda} P(O | \lambda)$.

Application: Train HMM from unlabelled data. **Solved by the Baum-Welch (Forward-Backward) algorithm.**

Computing $P(O|\lambda)$ — Brute Force vs. Forward

Naive approach: sum all paths

For observation $O = o_1 \dots o_T$ and N states, there are N^T possible hidden paths Q .

$$P(O|\lambda) = \sum_Q P(O, Q|\lambda)$$

Each path contributes:

$$P(O, Q) = \pi_{q_1} b_{q_1}(o_1) \prod_{t=2}^T a_{q_{t-1}q_t} b_{q_t}(o_t)$$

Cost: $O(2T \cdot N^T)$ — exponential in T .

For $N = 2$, $T = 3$: $2^3 = 8$ paths ✓

For $N = 45$ POS tags, $T = 20$: $45^{20} \approx 10^{34} \times$

Forward algorithm: $O(N^2 T)$

Define **forward variable**:

$$\alpha_t(j) = P(o_1, \dots, o_t, q_t = j | \lambda)$$

= probability of observing $o_1 \dots o_t$ and being in state j at time t .

Key trick

Reuse partial-path probabilities from step $t-1$ instead of re-computing from scratch. Same idea as dynamic programming for shortest paths.

Forward Algorithm — Recursion

Three steps:

1. Initialisation ($t = 1$):

$$\alpha_1(j) = \pi_j \cdot b_j(o_1)$$

2. Recursion ($t = 2, \dots, T$):

$$\alpha_t(j) = \left[\sum_{i=1}^N \alpha_{t-1}(i) a_{ij} \right] b_j(o_t)$$

3. Termination:

$$P(O \mid \lambda) = \sum_{j=1}^N \alpha_T(j)$$

Complexity: $O(N^2 T)$

Intuition for recursion:

- $\alpha_{t-1}(i)$: probability of reaching state i at $t-1$ while seeing $o_1 \dots o_{t-1}$
- a_{ij} : probability of transitioning $i \rightarrow j$
- $b_j(o_t)$: probability of emitting o_t from j
- Sum over all i : marginalise out the previous state

Sum not Max

Forward uses $\sum$ (marginalise). Viterbi uses $\max$ (optimise). Same trellis structure, different operator.

Forward Algorithm — Ice Cream Trellis ($O=313$)

State	$t=1$: eat 3		$t=2$: eat 1		$t=3$: eat 3	
	α_t	init	α_t	recursion	α_t	recursion
H	0.3200	$\pi_H b_H(3)$	0.0404	$(0.32 \cdot 0.6 + 0.02 \cdot 0.5) \cdot 0.2$	0.0235	$(0.0404 \cdot 0.6 + 0.069 \cdot 0.5) \cdot 0.4$
C	0.0200	$\pi_C b_C(3)$	0.0690	$(0.32 \cdot 0.4 + 0.02 \cdot 0.5) \cdot 0.5$	0.0051	$(0.0404 \cdot 0.4 + 0.069 \cdot 0.5) \cdot 0.1$

Sample ($t=2$, state C): $\alpha_2(C) = \left[\underbrace{0.32 \cdot 0.4}_{0.128} + \underbrace{0.02 \cdot 0.5}_{0.010} \right] \times b_C(1) = 0.138 \times 0.5 = \mathbf{0.069}$

Likelihood

$P(O \mid \lambda) = \alpha_3(H) + \alpha_3(C) = 0.0235 + 0.0051 = \mathbf{0.0286}$ (probability of observing sequence "313" under this HMM)

Three parameters [16]

$$\pi_i = P(\text{tag}_i \mid \text{START}) \quad a_{ij} = P(\text{tag}_j \mid \text{tag}_i) \quad b_i(w) = P(w \mid \text{tag}_i)$$

Running example — tags: {D, N, V}; words: {the, dog, cat, runs}

Transition matrix A

	D	N	V
START	0.80	0.15	0.05
D	0.05	0.90	0.05
N	0.15	0.10	0.75
V	0.30	0.10	0.60

Emission matrix B

	the	dog	cat	runs
D	0.90	0.02	0.03	0.05
N	0.01	0.50	0.45	0.04
V	0.01	0.05	0.04	0.90

First-order Markov Assumption

$P(t_i \mid t_1 \dots t_{i-1}) \approx P(t_i \mid t_{i-1})$ — tag depends *only* on the immediate predecessor. Real taggers learn A and B from millions of labelled sentences.

Viterbi variable $v_t(j)$

$$v_t(j) = \max_{q_1 \dots q_{t-1}} P(q_1 \dots q_{t-1}, q_t=j, o_1 \dots o_t \mid \lambda)$$

= probability of the **most likely** partial path ending in state j at time t .

Recursion:

$$v_t(j) = \left[\max_i v_{t-1}(i) a_{ij} \right] b_j(o_t)$$

Backpointer: $\psi_t(j) =$
 $\arg \max_i v_{t-1}(i) a_{ij}$

Forward vs. Viterbi

	Forward	Viterbi
Variable	$\alpha_t(j)$	$v_t(j)$
Operator	$\sum$	$\max$
Answers	$P(O \lambda)$	best Q
Backptr	no	yes (ψ)
Complexity	$O(N^2 T)$	$O(N^2 T)$

Both algorithms fill the **same trellis** shape. Only the combining operator differs: $\sum$ for likelihood, $\max$ for decoding.

State	t=1: eat 3		t=2: eat 1		t=3: eat 3	
	v_t	ψ_t	v_t	ψ_t	v_t	ψ_t
H	0.3200	—	0.0384	H	0.0128	C
C	0.0200	—	0.0640	H	0.0032	C

Sample ($t=2$, state C): $v_2(C) = \max(\underbrace{0.32 \cdot 0.4}_{0.128}, \underbrace{0.02 \cdot 0.5}_{0.010}) \times b_C(1) = 0.128 \times 0.5 = \mathbf{0.064}$ $\psi_2(C) = H$

Backtrace: **H** $\rightarrow$ **C** $\rightarrow$ **H**

Best at $t=3$: H (0.0128) $\psi_3(H) = C \Rightarrow$ at $t=2$: C $\psi_2(C) = H \Rightarrow$ at $t=1$: H $\checkmark$ Eat 3 on a **hot** day, 1 on a **cold** day, 3 on a **hot** day.

Viterbi Algorithm — Trellis Computation (POS)

Rahul Basu

What is NLP?

Text
Analytics
Framework

NLP Libraries

Text Pre-
processingFeature
ExtractionN-grams &
Language
Models

POS Tagging

Named Entity
RecognitionWord
Embeddings
& Word2VecText
Classification

Web Scrapping

Summary

Numerical
Exercises

Further

Sentence: "the dog runs" Recurrence: $\delta(j, t) = \max_i [\delta(i, t-1) \cdot a_{ij}] \cdot b_j(w_t)$

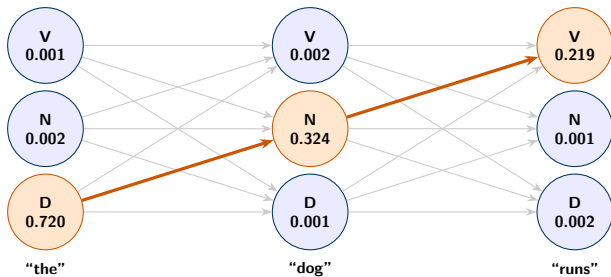
Tag	t=1: "the"		t=2: "dog"		t=3: "runs"	
	δ	ψ	δ	ψ	δ	ψ
D	0.7200	—	0.00072	D	0.00243	N
N	0.0015	—	0.3240	D	0.00130	N
V	0.0005	—	0.00180	D	0.2187	N

Sample ($t=2$, tag N): $\delta(N, 2) = \max(\underbrace{0.72 \cdot 0.90}_{0.648}, \underbrace{0.0015 \cdot 0.10}_{0.00015}, \underbrace{0.0005 \cdot 0.10}_{0.00005}) \times 0.50 = 0.648 \times 0.50 = \mathbf{0.324}$

Backtrack: D $\rightarrow$ N $\rightarrow$ V

"the" / **D** "dog" / **N** "runs" / **V** ✓ ψ traces the winning predecessor.

$P(D-N-V, O) = 0.80 \cdot 0.90 \cdot 0.50 \cdot 0.75 \cdot 0.90 = \mathbf{0.243}$



Shaded nodes = column-wise maximum δ . **Orange arrows** = backpointer path. Optimal sequence: **D N V**.

```
def viterbi(obs, pi, A, B):
    T, N = len(obs), len(pi)
    v = [[0.0]*N for _ in range(T)]
    psi = [[0] * N for _ in range(T)]
    # Initialise
    for s in range(N):
        v[0][s] = pi[s] * B[s][obs[0]]
        psi[0][s] = 0
    # Recursion
    for t in range(1, T):
        for s in range(N):
            scores = [v[t-1][r]*A[r][s]
                     for r in range(N)]
            v[t][s] =
                max(scores)*B[s][obs[t]]
            psi[t][s] =
                scores.index(max(scores))
    # Backtrack
    best = v[T-1].index(max(v[T-1]))
    path = [best]
    for t in range(T-1, 0, -1):
        best = psi[t][best]
        path.insert(0, best)
```

Three phases:

- 1 **Init:** seed column 1 with $\pi_j \cdot b_j(o_1)$
- 2 **Recursion:** fill each cell as max over previous column $\times$ transition $\times$ emission
- 3 **Backtrack:** start at argmax of last column; follow ψ backwards to recover path

Complexity

Time: $O(N^2 T)$ Space: $O(NT)$ for v and ψ
(N states, T time steps)

Backward Algorithm

Backward variable $\beta_t(i)$

$$\beta_t(i) = P(o_{t+1}, \dots, o_T \mid q_t=i, \lambda)$$

= probability of the *future* observations given we are in state i at time t .

Three steps:

- 1 **Init:** $\beta_T(i) = 1$ (all states)
- 2 **Recursion** ($t = T-1, \dots, 1$):

$$\beta_t(i) = \sum_{j=1}^N a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)$$

- 3 **Verification:** $P(O \mid \lambda) = \sum_j \pi_j b_j(o_1) \beta_1(j)$

Duality with Forward:

Forward $\alpha_t(j)$	Backward $\beta_t(i)$
Past obs + state j	State i + future obs
Left $\rightarrow$ right	Right $\rightarrow$ left
$P(o_1 \dots o_t, q_t=j)$	$P(o_{t+1} \dots o_T \mid q_t=i)$

Key use: state occupancy

$$P(q_t=j \mid O, \lambda) = \frac{\alpha_t(j) \beta_t(j)}{P(O \mid \lambda)}$$

This is $\gamma_t(j)$ — used in Baum-Welch.

EM algorithm for HMMs:

E-step: compute soft counts

State occupancy $\gamma_t(j)$:

$$\gamma_t(j) = \frac{\alpha_t(j) \beta_t(j)}{\sum_k \alpha_t(k) \beta_t(k)}$$

Transition count $\xi_t(i, j)$:

$$\xi_t(i, j) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{P(O | \lambda)}$$

M-step: re-estimate parameters

$$\hat{\pi}_j = \gamma_1(j)$$

$$\hat{a}_{ij} = \frac{\sum_t \xi_t(i, j)}{\sum_t \gamma_t(i)}$$

$$\hat{b}_j(v_k) = \frac{\sum_{t: o_t=v_k} \gamma_t(j)}{\sum_t \gamma_t(j)}$$

Convergence

Iterate E $\rightarrow$ M until $P(O|\lambda)$ stops increasing.
Guaranteed to reach a **local** maximum only —
use multiple random restarts.

Named Entity Recognition (NER)

NER identifies and classifies **named entity spans** in text: PERSON, ORG, GPE, DATE, MONEY, ...

IOB Tagging Scheme:

Apple	CEO	Tim	Cook	announced	results
B-ORG	O	B-PER	I-PER	O	O

- **B-** (Begin): first token of entity
- **I-** (Inside): continuation token
- **O** (Outside): not an entity

NER Approaches:

- NLTK: MaxEnt chunker on POS tags
- spaCy: transition-based neural model
- BERT-based: SotA on CoNLL-2003 [4]

Production Tip

NER entity-type distribution is a dataset health check. An unexpected spike in one type usually signals a domain mismatch between training data and inference data.

```
import spacy
from spacy import displacy

nlp = spacy.load("en_core_web_sm")
doc = nlp("Apple CEO Tim Cook announced
          "a $5B investment on
            Wednesday.")

# Entity access
for ent in doc.ents:
    print(ent.text, ent.label_,
          spacy.explain(ent.label_))

# Apple      ORG      Companies, agencies
# Tim Cook   PERSON   People
# $5B        MONEY    Monetary values
# Wednesday DATE     Dates/periods
# Render entity spans in Jupyter
displacy.render(doc, style="ent",
                jupyter=True)
# Use style="dep" for dependency tree
```

Entity span output (schematic):

Apple ORG CEO Tim Cook PER anno
a \$5B MONEY investment
on Wednesday DATE Useful doc

attributes:

- doc.sents — sentence iterator
- doc — token iterator (incl. pos_, dep_)
- doc.ents — named entity spans
- spacy.explain(label) — label description


```
from wordcloud import WordCloud
import matplotlib.pyplot as plt
from nltk.corpus import stopwords

stop = set(stopwords.words('english'))
stop.update(['said', 'would', 'could'])

wc = WordCloud(
    background_color='white',
    stopwords=stop,
    normalize_plurals=True, # film ==
    films
    collocations=False,
    max_words=200, random_state=42
).generate(corpus_text)

plt.figure(figsize=(10,4))
plt.imshow(wc, interpolation='bilinear')
plt.axis('off')
plt.savefig("wordcloud.png", dpi=150)
```

Parameter	Effect
normalize_plurals	Merges “film”/“films”
collocations	False = no auto bi-grams
max_words	Limit to top n terms
stopwords	Custom stop set

Use case

Word clouds are **exploratory** — they reveal dominant vocabulary at a glance but give no frequency scale.

Use **bar charts** for quantitative comparison; word clouds for presentations and stakeholder reports.

Rahul Basu

What is NLP?

Text
Analytics
Framework

NLP Libraries

Text Pre-
processingFeature
ExtractionN-grams &
Language
Models

POS Tagging

Named Entity
RecognitionWord
Embeddings
& Word2VecText
Classification

Web Scraping

Summary

Numerical
Exercises

Further

Era	Method	Representation	Key limitation
1950s–90s	BoW / TF-IDF [18]	Sparse, $ V $ -dim	No semantic proximity
2000s	LSA / LDA [2]	Dense topics	Static, no OOV
2013	Word2Vec [11]	Dense 100–300-dim	One vector per word
2014	GloVe [14]	Global co-occurrence	Same polysemy issue
2016	FastText [3]	Subword n-grams	Still static
2018+	BERT / GPT [4]	Contextual vectors	Computationally heavy

Word2Vec — CBOW and Skip-gram [11]

CBOW

Predicts **centre** word from surrounding context.

Context: $[w(t-2), w(t-1), w(t+1), w(t+2)] \rightarrow$

predict: $w(t)$

Average context embeddings before hidden layer.
Faster; better for frequent words.

Skip-gram

Predicts **context** words from centre word.

Centre: $w(t) \rightarrow$ **predict:** $[w(t-2) \dots w(t+2)]$

One centre generates multiple training pairs. Slower; better for **rare words**.

Self-supervised Objective

Neither architecture needs human labels. Both solve a **fake classification task** (predicting masked neighbours). The weight matrix that emerges from training *is* the word embedding matrix — geometry encodes semantics:

$$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$$

Word2Vec stores each word as a dense vector $\mathbf{v} \in \mathbb{R}^d$. Semantic similarity is measured by:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \in [-1, +1]$$

Interpretation:

- +1 — identical direction (synonyms)
- 0 — orthogonal (unrelated)
- -1 — opposite (antonyms)

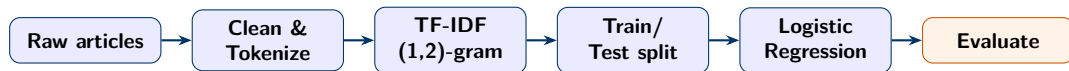
Example pairs (Reuters):

Pair	Cosine
oil — gas	≈ 0.82
bank — stock	≈ 0.71
oil — president	≈ 0.18

PCA Caveat

Projecting 100-dim vectors to 2D via PCA is **lossy**. Clusters visible in 2D are real, but close pairs in 2D may be distant in 100-dim space. Always verify similarity with `model.wv.similarity(w1,w2)` in full dimension.

End-to-End Pipeline — BBC News Classification



Parameter	Why
<code>min_df=3</code>	Drop hapax legomena; reduces noise
<code>ngram_range=(1,2)</code>	Bigrams capture “stock market”, “prime minister”
<code>sublinear_tf</code>	Replaces f with $1 + \log f$; compresses outlier counts
<code>C=5</code>	Weaker regularisation; high-dim TF-IDF space is noisy
5-fold CV	Estimates generalisation before touching held-out test set

Result

98.65% accuracy on BBC News (5 categories, $\approx 2,225$ articles). Expected — categories are lexically distinct. Real-world noisy data will score lower.

Reading Classification Metrics

Precision, Recall, F1

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}$$

$$F_1 = \frac{2PR}{P + R}$$

- High P : few false alarms
- High R : few missed instances
- F_1 : harmonic mean — punishes extreme imbalance between P and R
- Use **macro-F1** when class sizes differ

Confusion Matrix

- Rows = actual class
- Columns = predicted class
- Diagonal = correct predictions
- Off-diagonal reveals systematic confusion (e.g. tech ↔ business share vocabulary)

Top Features Per Class:

Logistic Regression coefficients map back to TF-IDF feature names. High-coefficient terms validate topic-meaningful learning and flag potential data leakage.

Web Scraping with BeautifulSoup

```
import requests
from bs4 import BeautifulSoup
import time

def scrape_wiki(topic):
    url = f"https://en.wikipedia.org/wiki/{topic}"
    r = requests.get(url, timeout=10)
    if r.status_code != 200:
        return None
    soup = BeautifulSoup(r.text,
                        'html.parser')
    paras = soup.find_all('p')
    text = ''
    for p in paras[:5]:
        text += p.get_text(strip=True) + '\n'
    time.sleep(1) # politeness
    return text
```

Key steps:

- requests.get() — HTTP GET, check status 200
- BeautifulSoup — parse HTML tree
- soup.find_all('p') — extract paragraphs
- p.get_text() — strip all HTML tags
- time.sleep(1) — politeness; avoid IP bans

Ethics & Legality

Always check robots.txt before scraping. Many sites prohibit automated access. Use official APIs where available.

Text Extraction — PDF & Images (OCR)

PDF (pdfminer.six)

```
# One-liner (pdfminer >= 20201018)
from pdfminer.high_level import
    extract_text
text = extract_text("report.pdf")

# Fine-grained control
from pdfminer.pdfinterp import (
    PDFResourceManager,
    PDFPageInterpreter)
from pdfminer.converter import
    TextConverter
from pdfminer.layout import LAParams
from io import StringIO

rsrcmgr = PDFResourceManager()
output = StringIO()
device = TextConverter(
    rsrcmgr, output,
    laparams=LAParams())
interp =
```

Images / Scanned PDFs (pytesseract)

```
from PIL import Image
import pytesseract

# Windows: set tesseract path
pytesseract.pytesseract.tesseract_cmd
    = (
        r'C:\Program
            Files\Tesseract-OCR',
        r'\tesseract.exe')

img = Image.open("scan.png")
text =
    pytesseract.image_to_string(img)

# Multi-language (Hindi example)
text_hi =
    pytesseract.image_to_string(
        img, lang='hin')
```


Sentiment Analysis — TextBlob

```

from textblob import TextBlob

reviews = [
    "The film was absolutely amazing!",
    "Terrible acting, complete waste.",
    "An interesting but slow-paced
      story."]

for text in reviews:
    blob = TextBlob(text)
    p = blob.sentiment.polarity
    s = blob.sentiment.subjectivity
    print(f"P={p:+.2f}  S={s:.2f}
          {text[:30]}")

# P=+0.80  S=0.75  The film was
    absolutely...
# P=-0.80  S=0.90  Terrible acting,
    complet...
# P=+0.20  S=0.55  An interesting but
    slow-...

```

Score

Meaning

Polarity

[-1, +1]

-1 = very negative

0 = neutral

+1 = very posi-
tive

Subjectivity

[0, 1]

0 = objective fact

1 = personal
opinion

Lexicon limitation

TextBlob (PatternAnalyzer) sums word polarities independently. “not bad” scores ≈ 0.0 instead of $+0.35$ — negation is ignored. For production: use a fine-tuned transformer (e.g. `cardiffnlp/twitter-roberta-base-sentiment`).

Session Summary

Core concepts covered:

- 1 NLP definition, challenges, applications
- 2 Text Analytics lifecycle (iterative pipeline)
- 3 NLTK vs spaCy — when to use which
- 4 Pre-processing: lowercase, spell-correct, emoji, URL, stopwords, stem/lemmatize (with POS map)
- 5 NLTK corpora: Reuters, Movie Reviews; CountVectorizer & DTM; TF-IDF vectorizer
- 6 N-grams: bigram LM, Laplace smoothing, sklearn extraction
- 7 POS tagging: NLTK/spaCy code, HMM matrices, Viterbi trellis
- 8 NER: IOB scheme, spaCy displacy, entity iteration
- 9 Word cloud, PDF extraction (pdfminer), OCR (pytesseract)
- 10 TextBlob sentiment (polarity/subjectivity); Word2Vec; BBC pipeline & metrics

Key takeaways:

- Always POS-tag before lemmatizing — default NOUN silently corrupts verbs
- TF-IDF is a soft stopwords filter built into the maths
- `min_df` trims hapax; sparse CSR is $300\times$ smaller than dense
- Viterbi is DP on the product semiring — same pattern as shortest path
- TextBlob fails on negation; transformer models are the production baseline
- 98% accuracy on BBC is easy; real NLP is much harder

Next session:

Advanced vectorisation — GloVe, FastText, subword models, contextual embeddings (ELMo,

Exercise Set A — HMM & Viterbi

Use the transition and emission matrices from the HMM slides for all questions.

- ① [1 pt] **Joint probability.** Compute $P(\mathbf{t}, \mathbf{w})$ for the tag sequence D-N-V on “the dog runs”:

$$P = \pi(t_1) \cdot b_{t_1}(w_1) \prod_{i=2}^3 a_{t_{i-1}t_i} \cdot b_{t_i}(w_i)$$

- ② [1 pt] **Initialisation.** For the sentence “cat runs”, compute $\delta(D, 1)$, $\delta(N, 1)$, $\delta(V, 1)$. Which tag wins at $t=1$? Does this match linguistic intuition?
- ③ [2 pts] **Full Viterbi.** Complete the trellis for “cat runs” and state the optimal tag sequence.
Hint: $\delta(j, 2) = \max_i [\delta(i, 1) \cdot a_{ij}] \cdot b_j(\text{runs})$
- ④ [1 pt] **Linguistic insight.** $a_{N,V} = 0.75$ is the largest off-diagonal value in A . What English grammatical pattern makes noun $\rightarrow$ verb transitions so probable?

- ① [2 pts] **TF-IDF**. Mini corpus ($N = 3$): D1: “cat sat mat”, D2: “cat sat hat”, D3: “dog bit cat”.
- (a) Compute $\text{IDF}(t) = \log \frac{N+1}{n_t+1} + 1$ for the terms *cat*, *sat*, *mat*.
 - (b) Rank by IDF value. Which single term best discriminates D1 from D2? Explain why the other terms fail.
- ② [2 pts] **Bigram language model**. Corpus: “the cat sat. the cat sat on the mat.”
- (a) Build a bigram count table for {the, cat, sat, on, mat} (treat . as a sentence boundary, not a token).
 - (b) Apply Laplace smoothing ($|V| = 5$); compute $P_L(\text{cat} \mid \text{the})$ and $P_L(\text{mat} \mid \text{on})$.
- ③ [1 pt] **Thought question**. Laplace smoothing assigns $P(\text{sat} \mid \text{the}) > 0$ even though the bigram “the sat” never appears in training. Is this a *feature* (generalisation) or a *bug* (noise)? Justify in one sentence.

Further Reading & Online Resources

Textbooks

- Jurafsky & Martin — *Speech and Language Processing* (3rd ed., free online) [8]
- Manning & Schütze — *Foundations of Statistical NLP* [10]
- Goldberg — *Neural Network Methods for NLP* [5]
- Bird et al. — *Natural Language Processing with Python* [1]

Classic Papers

- Porter (1980) — Suffix-stripping stemmer [15]
- Miller (1995) — WordNet [12]
- Jones (1972) — IDF weighting [7]
- Shannon (1948) — Mathematical theory of communication [19]

Modern Papers

- Mikolov et al. (2013) — Word2Vec [11]
- Pennington et al. (2014) — GloVe [14]
- Bojanowski et al. (2017) — FastText [3]
- Devlin et al. (2019) — BERT [4]
- Lafferty et al. (2001) — CRFs [9]

Online

- SLP3 draft (Stanford)
- HuggingFace NLP Course
- spaCy documentation
- NLTK Book (free)
- Mikolov's Word2Vec code

References I



S. Bird, E. Klein, and E. Loper.
Natural Language Processing with Python.
O'Reilly Media, 2009.
Free online at <https://www.nltk.org/book/>.



D. M. Blei, A. Y. Ng, and M. I. Jordan.
Latent Dirichlet allocation.
Journal of Machine Learning Research, 3:993–1022, 2003.



P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov.
Enriching word vectors with subword information.
Transactions of the Association for Computational Linguistics, 5:135–146, 2017.



J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova.
BERT: Pre-training of deep bidirectional transformers for language understanding.
In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.



Y. Goldberg.
Neural Network Methods for Natural Language Processing.
Morgan & Claypool, 2017.



M. Honnibal, I. Montani, S. Van Landeghem, and A. Boyd.
spaCy: Industrial-strength natural language processing in python, 2020.



K. S. Jones.
A statistical interpretation of term specificity and its application in retrieval.
Journal of Documentation, 28(1):11–21, 1972.

References II



D. Jurafsky and J. H. Martin.

Speech and Language Processing.

3rd (draft) edition, 2024.

Available at <https://web.stanford.edu/~jurafsky/slp3/>.



J. D. Lafferty, A. McCallum, and F. C. N. Pereira.

Conditional random fields: Probabilistic models for segmenting and labeling sequence data.

In *Proceedings of the 18th ICML*, pages 282–289, 2001.



C. D. Manning and H. Schütze.

Foundations of Statistical Natural Language Processing.

MIT Press, 1999.



T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean.

Distributed representations of words and phrases and their compositionality.

In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26, 2013.



G. A. Miller.

WordNet: A lexical database for English.

Communications of the ACM, 38(11):39–41, 1995.



OpenAI.

GPT-4 technical report.

Technical report, OpenAI, 2023.

arXiv:2303.08774.

References III



J. Pennington, R. Socher, and C. D. Manning.

GloVe: Global vectors for word representation.

In *Proceedings of EMNLP*, pages 1532–1543, 2014.



M. F. Porter.

An algorithm for suffix stripping.

Program, 14(3):130–137, 1980.



L. R. Rabiner.

A tutorial on hidden Markov models and selected applications in speech recognition.

Proceedings of the IEEE, 77(2):257–286, 1989.



G. Salton and C. Buckley.

Term-Weighting Approaches in Automatic Text Retrieval, volume 24.

1988.



G. Salton, A. Wong, and C. S. Yang.

A vector space model for automatic indexing.

Communications of the ACM, 18(11):613–620, 1975.



C. E. Shannon.

A mathematical theory of communication.

The Bell System Technical Journal, 27:379–423, 623–656, 1948.